

Number of Processors	Time Taken (seconds)	Speedup (T1 / Tn)
1 Processor	0.026806	1.00x
2 Processors	0.013975	1.92x
4 Processors	0.018229	1.47x
8 Processors	0.014052	1.91x

Observation and Analysis The parallel implementation demonstrates a near-linear speedup when scaling from 1 to 2 processors (1.92x). However, the performance degrades and the speedup drops to 1.47x when using 4 processors, before recovering slightly at 8 processors.

This bottleneck occurs due to the hardware limitations of the underlying AWS EC2 instance environment. Since the instance is equipped with a limited number of virtual CPUs (typically 2 vCPUs for standard student tiers), requesting 4 or 8 MPI processes forces the operating system to perform heavy context switching. The overhead of the CPU constantly swapping between the active processes outweighs the benefits of parallelization, resulting in higher execution times for processor counts that exceed the physical core count of the machine.

Exercise 5: Source and Destination Mismatch

- **Observation:** When modifying the program so the source and destination ranks do not match (e.g., Rank 1 sends to Rank 2, but Rank 3 is waiting to receive from Rank 1), the program results in a **deadlock**. Rank 1 blocks indefinitely waiting for the destination to receive the message, while Rank 3 blocks indefinitely waiting for a message that was routed elsewhere.
- **Solution:** The code was rewritten using Buffered Send (MPI_Bsend) to prevent blocking behavior. The updated code is available in the repository under Exercise5.